

Approach Indicator

Version 6.2

Contents

Approach Indicator	1
The Project	1
Two Sketches: ApproachIndicator and ResistTest	1
Sketch: ApproachIndicator 6.2.....	2
Features	2
The software provides:	2
The Design.....	2
Alerts	2
Approaches	3
Placing Sensors.....	3
Sensors-Analog.....	3
Sensors-Digital	4
Board Schematic	4
MRCS Board	5
Sketch: ResistTest	6
Features	6
Definitions:.....	6

The Project

Many railroad interlocking towers had alarm bells to alert the tower operator that a train was approaching. On a large club railroad where I volunteer there are train order offices located under the layout. We set up a closed-circuit camera system so that the agent/operator can see the approaches to their town. We also wanted to implement alerts when a train approached.

We decided to implement an Arduino-based solution to the problem. Arduinos are a low-cost, highly programmable solution for analog and digital sensor-based projects.

Two Sketches: ApproachIndicator and ResistTest

There are two sketches involved. ApproachIndicator is the primary sketch which provides the approach notification function. ResistTest helps confirm and set the sensitivity of the photoresist devices if used for sensing.

Sketch: ApproachIndicator 6.2

Features

The software provides:

- Up to eight approaches of location sensors per Arduino Nano. Four using analog sensors and four using digital sensors.
- Each approach consists of 2 analog or digital sensors, distant and home.
- Each approach drives an individual or shared alert.
- Alert duration is timed and may be cancelled (acknowledged) by an optional cancel signal (button).
- Analog sensors are automatically and periodically recalibrated.

The Design

The design is based on having two sensors at each approach. These *home* and *distant* sensor sets for the approach are monitored to determine the direction of travel of the train. If the distant sensor is triggered and subsequently the home sensor is triggered the train is approaching and the alert will be triggered.

The sketch for the Arduino supports both analog and digital detectors. Analog (photo-sensitive) sensors are automatically calibrated and the sensitivity is configurable.

Alerts

The software generates alerts via the definitions tying an approach definition to an alert definition. The alert definition pins may drive a relay for activating an audio or visual notification. Multiple approaches may be directed to the same alert if required. The alert duration in seconds is set here, as is an optional port for a *clear* button. After the alert is triggered, signaling *clear* will stop the alert. Note: Leave

DontRepeatMS & AlertEnd values at zero.

Definition:

```
typedef struct {
    // for buzzer or light -- Arduino address digital
    int SndPort;
    // Duration of light or sound -- 0 means keep on until cleared
    int AlrtSecs;
    // Arduino port for reset button if required
    int RstPort;
    unsigned long DontRepeatMS; // expire time for blocking repeat alert in seconds
    unsigned long AlertEndMs;   // expire time for alerts
}AlrtDef;
```

Example: Two alerts defined, 60 seconds, with *clear* button defined.

```
AlrtDef Alrts [AlrtCnt] = {  
    // signal port, duration secs, clear port, (DontRepeatMS, AlertEnd)  
    3, 60, 5,      0,0, // Norwich  
    2, 60, 4,      0,0 // Stafford  
}; // Alrts
```

Make sure that these pins don't overlap with the digital sensor pins if used.

Approaches

The software monitors each approach.

Definition:

```
// DEFINE each approach  
typedef struct {  
    // SnsrType TypeAlog TypeDigt  
    // both sensors must be the same type  
    int SnsrType;  
    // Sensor addresses: Distant - Home  
    int SnsrLst [2];  
    // index into Alert table  
    int AlrtIdx;  
}ApproDef;
```

Example: Four approaches defined, tied to two different alerts.

```
// define the approaches  
ApproDef Appro [ApproCnt] = {  
    // define each approach  
    // Type Distant Home Alert  
    // *** Stafford approach alert 0  
    TypeAlog, A0, A1, 0, // Stafford left  
    // *** Stafford approach alert 0  
    TypeAlog, A2, A3, 0, // Stafford right  
    // *** Norwich approach alert 1  
    TypeAlog, A4, A5, 1, // Norwich left  
    // *** Norwich approach alert 1  
    TypeAlog, A6, A7, 1 // Norwich right  
}; // Appro  
// END Norwich - Stafford definitions
```

Placing Sensors

The sensors are placed in pairs, each pair having a *home* and a *distant* sensor. Each sensor pair must be of the same type, either analog or digital.

Track diagram for a typical tower.



Sensors-Analog

Light detecting devices, either phototransistors or photoresistors may be used. They are connected to the Arduino's analog ports. The design calls for pull-down resistors on the ground side for each sensor.

The software monitors the drop in signal between when the sensor is not covered and when it is covered by rolling stock. There is a defined parameter which sets the threshold for the sensitivity of the sensor, i.e. unless the sensor drops the defined percentage the sensor is considered unoccupied. Refer to the ResistTest sketch for setting this value.

```
// Sensitivity - alert only if analog sensor drops this % from normal
#define Sensity 70
```

The sensors may be affected by LED lighting so the sketch integrates readings over a power cycle, i.e. 34 milliseconds for the 60 hertz mains. The numbers below control the integration loop and are in milliseconds.

```
// analog sampling controls - needed to average light flicker
// length of sample integration in millisecs
// 34 ms for 60 cycle lighting mains
// 40 ms for 50 cycle lighting mains
#define LoopMS 34
```

The software automatically calibrates the sensors. There is a periodic recalibration for the analog sensors in case sunlight or other conditions change the light hitting the sensors.

```
// reset sensors every RecalMts
#define RecalMts 10
```

Sensors-Digital

A digital signal (HIGH/LOW) can come from an infrared detector or some such. The define below controls how the signal is treated.

```
// if digital track sensors use low for occupied, true
#define DigInvert false
// #define DigInvert true
```

Sensor setup: When installing or debugging an installation, there is a need to confirm that the Arduino was detecting the sensor properly. The on-board LED 13 may be for this purpose. A 0 (zero) will disable this feature. In addition, refer to the ResistTest sketch (below) for additional help during the install process.

```
// if SnsrActiveLED is non-zero, light the LED indicated if any sensor is active
#define SnsrActiveLED 13
// #define SnsrActiveLED 0
```

Testing using a train with skeleton logging cars and a very short logging caboose caused the sensors to alternate between clear and occupied while the train was passing. There is a timer to hold a sensor *occupied* (in software) to control wild swings caused by skeleton cars or couplings.

```
// hold occupied status for this time to minimize bounce (false clear)
// - for couplers, skeleton cars - in milliseconds
#define HoldOccMs 2000
```

Board Schematic

There is a separate document ApproachIndicatorSchematic.docx which shows the layout of my wire-wrap prototype board.

MRCS Board

Model Railroad Control Systems offers a PCB for this project. Contact them at
<https://www.modelrailroadcontrolsystems.com/>

Sketch: ResistTest

Features

This sketch monitors the analog ports and reports via the Serial Monitor for each pin:

- Pin number
- Highest value
- Current average value
- Percentage drop from the highest value

The report is useful to confirm each port, its connection, and its change in resistance when covered. The percentage drop value is useful for setting the Sensity setting in the ApproachIndicator and other sketches.

Definitions:

List here the ports to be monitored:

```
#define NumS 8
const int Snsrs[NumS] = {A0,A1,A2,A3, A4,A5,A6,A7}; // analog sensors
```

Other defines:

```
const int MSLoop = 33;           // interval for averaging data
const int DlyInLoop = 0;         // MS delay within the data collection - keep at 0
const int DlyOutLoop = 5000;     // delay between reports
const int RecalibrateMts = 1;    // duration before recalibrating in minutes
```